

COMP 356
Programming Language Structures
Notes for Chapter 5 of *Concepts of Programming Languages*
Names, Types and Scopes

Some definitions:

- a **name** is a string of characters (a word) that represents a program entity (variable, type, subprogram, ...)
- a **reserved word** is a word that can not be a user-defined name. For example, `if`, `else` and `for` are reserved words in C++.
- a **keyword** is a word that has a special meaning, but that can be redefined by the programmer. For example, `INTEGER` and `REAL` are type names in FORTRAN, but can also be used as variable names.

Other authors use keyword to mean both keywords and reserved words.

Attributes of variables:

- name (identifier)
- address (l-value)
- value (r-value)
- type
 - range of possible values for the variable (set theory)
 - “tag” or “classification” of the variable
- lifetime - the time during which the variable is bound to (associated with) a particular address
- scope - the region of the program from which the variable is visible (can be accessed)

1 Bindings and Lifetimes

More definitions:

- a **binding** is an association between two entities. For example, a variable is bound to its type, its address, ...
- a **static binding** occurs before runtime and doesn't change during execution. Examples:
 - bindings of values to constants in C
 - bindings of function calls to function definitions in C
- a **dynamic binding** occurs or changes at runtime. Examples:
 - bindings of values to variables
 - bindings of member function calls to virtual member function definitions in C++
 - all bindings of messages (method calls) to methods in Java

1.1 Type Binding

One important kind of binding is **type binding** - binding of types to variables. Type binding can be static or dynamic.

Static type binding:

- is the usual approach and is used in C, C++, Java, Pascal, ...
- can be done in several ways:
 - explicitly, through variable declarations
 - implicitly, through rules or conventions. For example, any variable that starts with the letter 'i' in FORTRAN is an **INTEGER** unless declared otherwise
 - inferred, using type inference. For example, SML deduces the types of the parameters of a function based on how the parameters are used in the function's body

Dynamic type binding:

- means that the type of a variable can change at runtime
- often means that variables are not declared
- is found mostly in older (LISP, BASIC, ...) and scripting (Perl, TCL, ...) languages

The advantage of dynamic type binding is flexibility. Disadvantages include:

- reduced error checking
 - all type checking is dynamic (code that isn't executed isn't type checked)
 - assigning a value of the "wrong" type to a variable changes the type of the variable and generates no errors (until later ...)
- increased cost
 - dynamic type checking
 - the type of each variable must be stored with it

Using an interpreter "hides" these costs.

1.2 Lifetimes

Still more definitions:

- the **lifetime** of a variable is the time during which the variable is bound to a particular address
- **allocation** is binding a variable to a memory location (an address)
- **deallocation** is returning a memory cell to free memory after it is unbound from a variable

Variables can be placed in one of four categories depending on their lifetimes:

1. static variables

- lifetime = entire program execution
- examples:
 - global variables
 - local variables declared **static** in C and C++
 - * maintain their values between function executions (history sensitive)
 - * this requires statically allocated storage

* useful for doing counting etc. without introducing global variables

2. stack-dynamic variables

- lifetime = (roughly) execution of the block they are declared in
- examples: nonstatic local variables in functions and blocks

elaboration of a declaration refers to allocating storage and binding it to the variable in a declaration. In languages that allow variable declarations after the beginning of a block (Java, C++), elaboration of such variables may occur at the beginning of the nearest enclosing block, or at the point where the variable is declared (implementation dependent). The lifetime of a stack-dynamic variable is from elaboration time until the end of the execution of the block. Hence, the lifetime of such a variable may be less than the execution of the block.

3. explicit heap-dynamic variables

- explicit heap-dynamic variables are memory cells explicitly allocated from the heap
- examples: memory cells allocated using `new` in Java and C++
- often anonymous (unnamed)
- lifetime = allocation time until explicit deallocation or garbage collection
- example: in C++, the lifetime of such a variable is from the time it is allocated (using `new`) until it is deallocated (using `delete`)

4. implicit heap-dynamic variables

- implicit heap-dynamic variables are variables for which storage isn't allocated until the variable is assigned to, and the storage associated with the variable can change with every assignment to it
- lifetime = from one such assignment to the next
- examples: strings and arrays (hashes) in Perl, arrays in JavaScript
- implicit heap-dynamic variables are extremely flexible, but also expensive to use

2 Type Checking

Even more definitions:

- **Type checking** is checking that the types of operands match (are compatible with) the types expected by operators. Operators include arithmetic, relational and boolean operators, = (assignment), user defined and built-in functions.
- Two types are **compatible** if they match exactly or one can be implicitly converted (**coerced**) to the other. For example:

```
float f = 3;
```

type checks in C++ because `int` is coerced to `float` in this context.

- A **type error** is an application of an operator to an operand of an incompatible type.
- A language is **strongly typed** if type errors are always detected (statically or dynamically). An alternative (weaker) definition used by some authors is that a language is strongly typed if the type of each name can always be determined at compile time and can't change at runtime.

C++ is not a strongly typed language because:

- calls of unchecked functions are not type checked. An unchecked function is a function with a variable number and type of parameters such as `printf()`.
- arbitrary pointer casts are allowed. For example:

```
int *i = new int;
*i = 3;
float *f = (float *) i; // type error
cout << *f; // prints "junk"
```

- union types are included. For example:

```
union {
    int i;
    double d;
} u;

u.i = 3;
cout << u.d;
```

Java, Ada and SML are strongly typed languages.

The type compatibility rules of a language have a huge impact on the design of the language and also on the reliability of programs written in the language. For example:

- the type compatibility rules of C++ are extremely flexible, because C++ does many coercions. Hence, some type errors such as using an integer division instead of a floating point division are frequently not detected.
- the type compatibility rules of Ada are extremely rigid, because Ada does no coercions. This makes Ada a very reliable programming language, but forces programmers to do many explicit type casts. For example, the programmer must use an explicit cast to add an integer and a floating point number.

The fundamental rule of type checking: if f is a function from type A to type B , i.e. $f:A \rightarrow B$, and f is applied to an argument of type A , the resulting expression is of type B .

For example, $+: \text{int} \times \text{int} \rightarrow \text{int}$. Since $(3, 4) \in \text{int} \times \text{int}$, $3 + 4$ is of type int .

Type compatibility rules are used in the following contexts. Given $f:A \rightarrow B$ and the call:

```
x = f(E);
```

- is the type of E compatible with type A ?
- is the type of x compatible with type B ?

Three different type compatibility rules (or variants) are commonly used:

- structural type compatibility
- name type compatibility
- declaration type compatibility

2.1 Structural Type Compatibility

Informally, two types are compatible under structural type compatibility if they “have the same structure” in memory.

Formally, two type expressions are structurally compatible iff:

1. they are the same type name
2. they are formed by applying the same **type constructor** to structurally compatible types
3. after a type declaration like:

```
type S1 = S2;
```

or in C++:

```
typedef S2 S1;
```

`S1` and `S2` are structurally compatible types.

A **type constructor** is an operator that builds new types. For example, in C++ the type constructors include: `[]`, `struct`, `class`, `union` and `*`.

Consider the following type declarations:

```
typedef char atype;
typedef char s1[10];
typedef atype s2[10];
typedef struct {char c; s2 s;} s3;
typedef struct {atype c; s1 s;} s4;
```

After these declarations:

- types `char` and `atype` are structurally compatible
- types `s1` and `s2` are structurally compatible
- types `s3` and `s4` are structurally compatible

2.2 Name Type Compatibility

Formally:

1. a type name is compatible only with itself
2. no constructed type (expression containing a type constructor) is compatible with any other

Name type compatibility is much more restrictive than structural type compatibility. For example, from the declarations in the previous section, none of `char`, `atype`, `s1`, `s2`, `s3` or `s4` is name type compatible with any other type in the list. As another example:

```
int *i1;
int *i2;
```

After these declarations, the types of `i1` and `i2` are not name type compatible. In a language that uses name type compatibility, variables `i1` and `i2` could not be compared or assigned to each other.

2.3 Declaration Type Compatibility

Formally:

1. a type name is compatible with itself and with any type name it is (transitively) declared equivalent to
2. no constructed type (expression containing a type constructor) is compatible with any other

For example:

```
typedef char s1;  
typedef s1 s2;
```

After these declarations, the types `char`, `s1` and `s2` are all declaration type compatible.

As another example:

```
int *i1;  
int *i2;
```

After these declarations, the types of `i1` and `i2` are not declaration type compatible, because they are constructed types.

Comparisons:

- structural type compatibility is:
 - the most flexible
 - the most difficult to implement (requires a recursive procedure to compare type expressions)
- name type compatibility is:
 - the easiest to implement (just use `strcmp` on the type names)
 - the most restrictive

For example, under name type compatibility, the following is a type error:

```
void foo(int *i) { ... }  
  
int *i2;  
foo(i2); // type error here
```

because 2 constructed types are not name compatible. The same situation occurs for array types.

In languages such as early versions of Pascal that use name type compatibility, the fix is to declare type names globally and use them rather than constructed types. For example:

```
typedef int *iptr;  
  
void foo(iptr i) { ... }  
  
iptr i2;  
foo(i2); // OK
```

The only Java constructs that create user-defined types are classes and interfaces. Java uses name compatibility for these types, and structural type compatibility for arrays.

The type compatibility rules for C and C++ are:

- declaration type compatibility for structs, classes and unions
- structural type compatibility for all other types

The text claims (incorrectly) that C++ uses only name compatibility.

Why declaration type compatibility for structs? Consider:

```
typedef struct foo {int info; struct foo *next;} footype;
typedef struct bar {int info; struct bar *next;} bartype;
```

Are **footype** and **bartype** structurally compatible? The **info** fields are, and the **next** fields are if **footype** and **bartype** are structurally compatible ...

In the presence of **circular** or **recursive** types (types defined in terms of themselves), structural type compatibility is **undecidable**. This means that no algorithm for solving this problem exists. In C++, the only mechanisms for creating circular types are structs and classes.

Ada uses a variant of name type compatibility – subtypes are compatible with the types they are constructed from. All other types use name type compatibility (except unconstrained types).

3 Scope

More definitions:

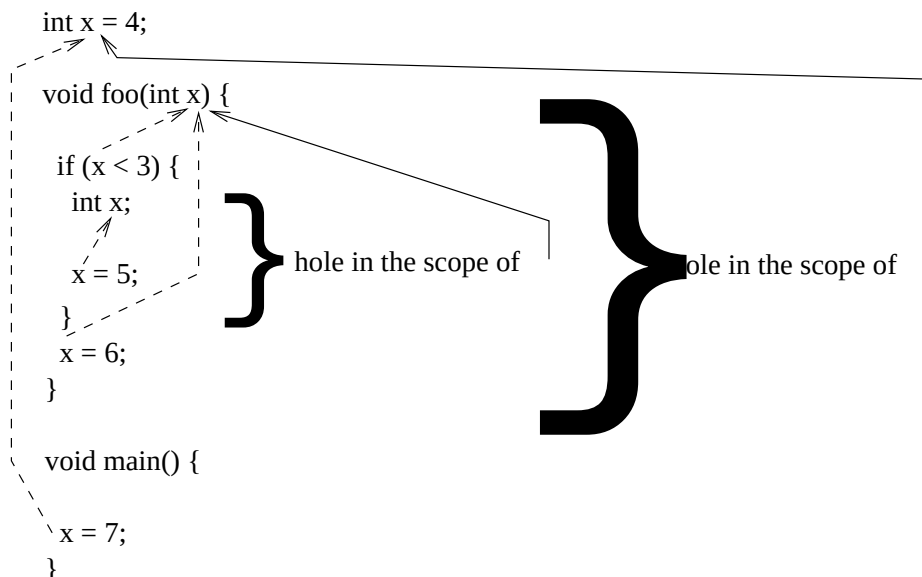
- the **scope** of a variable is the region of the program text in which the variable is visible. The scoping rules of a language match each occurrence of a variable name with a declaration of that name.
- a variable is **visible** if it can be referenced (used, referred to, ...)
- a variable is **local** to a block if it is declared there
- a variable that is visible in a block but not local to it is a **nonlocal variable** of that block

Scope can be **static** or **dynamic**.

3.1 Static Scope

- names (variable references) are bound to declarations statically (at compile time)
- this can be done using only the program text
- the declaration of the name in the closest enclosing block of the program is used
- interesting behavior occurs if blocks (and their associated scopes) can be nested

Consider the following C++ program:



The dashed arrows match **bound occurrences** of the variable name `x` to the appropriate declarations (**binding occurrences**) for static scope.

A redeclaration of a name in a nested scope creates a **hole in the scope** of that name in the outer scope, because the outer declaration is not visible in the nested scope.

If a variable is declared in the middle of a block, its scope is from the declaration to the end of the (nearest enclosing) block.

Almost all modern programming languages use static scope - it is more efficient and usually easier to understand than dynamic scope.

3.2 Dynamic Scope

Under dynamic scope, a name (variable reference) is bound to the most recently seen declaration of that name (at runtime).

For example, consider the following program.

```
// program checkscope
#include <iostream>

int x = 3;

void printx(void) { std::cout << x << std::endl; }

void foo(void) {
    int x = 4;
    printx();
}

int main() {
    printx();
    foo();
}
```

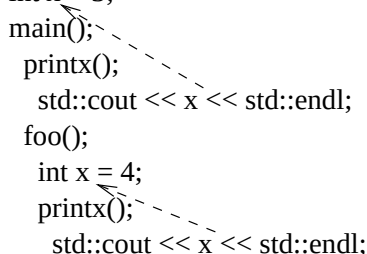


Again, the dashed arrow shows the binding of the nonlocal `x` in `printx()` to the global declaration of `x` under static scope. Hence, when this program is run under static scope as in C++, the output is:

```
3
3
```

Under dynamic scope, the binding of variable occurrences to declarations is determined by the sequence of statements executed, as follows:

```
int x = 3;
main();
printx();
    std::cout << x << std::endl;
foo();
    int x = 4;
    printx();
        std::cout << x << std::endl;
```



The dashed arrows now show the binding of `x` to the appropriate declarations for dynamic scope. Hence, the output under dynamic scope is:

```
3
4
```

Dynamic scope is implemented by searching down the runtime stack until a declaration of the name is found. In particular, once the execution of a subprogram is finished, declarations in that subprogram will not be used.

Note that dynamic scope forces typechecking to be dynamic, as the binding of variable occurrences to their declarations can not be determined at compile time.

Dynamic scope is used in APL and early versions of LISP. Modern LISPs have both static and dynamic scope, but the default is static scope.

3.3 Final Notes

- the lifetime and (static) scope of a variable are often different
 - a variable's lifetime includes holes in its scope
 - static local variables in C and C++ have local scope, but their lifetime is the entire program execution
- Java, C and C++ permit variables to be initialized when they are declared
 - stack dynamic variables are dynamically allocated and initialized
 - static variables (including globals) are statically allocated and initialized

Hence, static local variables are initialized only once, regardless of how many times the blocks they are declared in are executed. For example, in:

```
void foo(void) {
    static int x = 3;

    ...
}
```

`x` is initialized once, regardless of how often `foo()` is called.

- most modern languages provide **named constants**
 - C
 - * example: `#define PI 3.14159`
 - * only has **manifest constants**, which can not be initialized to the value of a variable
 - C++
 - * example:

```
double halfPI = 1.5707;
const double PI = 2 * halfPI;
```
 - * can be initialized using variables
 - Java
 - * example:

```
double halfPI = 1.5707;
final double PI = 2 * halfPI;
```
 - * **final** variables can only be assigned once, but this need not occur in the declaration
 - * **blank finals** are fields initialized in the constructor to create immutable objects. Examples: fields of the built-in `String` and `Integer` classes